

Module 2: Data Modeling (ER)

2.1 Entities, Attributes & Relationships — Core • 14 min

Hook

Your college gives you a blank page and one sentence:

"Build us a system that tracks who studies what."

No tables. No columns. Just that one line. Where does your pen go first?

Why + What

Most people jump straight to writing tables. That's a mistake. You don't know yet what things exist in this story, or how they connect. If you guess wrong, you find out three months later — when there's already real data inside, and fixing the design means moving all of it.

Drawing it on paper first takes an hour. Fixing it later takes a weekend.

So before tables, you draw a picture. That picture is called an ER model.

An ER (Entity–Relationship) model is a drawing of the real world before it becomes tables. It has three parts: entities (the things you store data about), attributes (the facts about each thing), and relationships (how the things connect).

Three words. Three shapes. That's all you need.

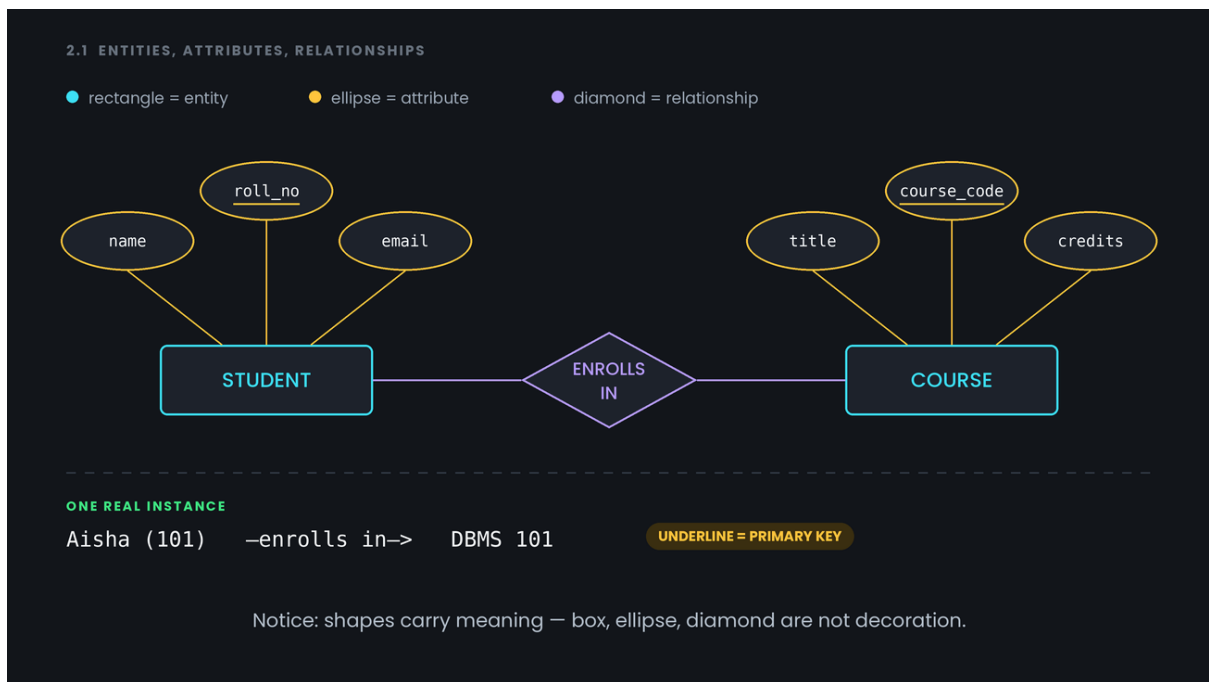
How It Works (with the easiest example)

Let's pull that one sentence apart.

1. **Find the nouns.** *Student* and *Course*. Each one is a real thing you want to store data about. So each becomes an **entity**. In the drawing, an entity is a **box**.
2. **Describe each noun.** A student has a name, a roll number, an email. A course has a code, a title, credits. These facts are **attributes**. They belong to an entity — they're not things on their own. An attribute is an **ellipse**.
3. **Pick the one that's never repeated.** Which of Aisha's attributes is unique to her? Not her name — another student could also be called Aisha. Her **roll number**. That one is the **key attribute**, and we show it by **underlining** it.
4. **Find the verb.** "Studies." A student *enrolls in* a course. That's a **relationship**. It sits *between* the two boxes, not inside either one. A relationship is a **diamond**.
5. **Test it with one real example.** Aisha (101) enrolls in DBMS 101. Every word of that fits somewhere in the drawing, and nothing in the drawing is left unused. The model works.

Pause here. Prof. Rao is about to join this model. Before you read on: is he an entity, an attribute, or a relationship? Say why in one line.

He's an **entity**. He has his own facts about him — name, department — and he exists whether or not any particular course does. If you made him just an attribute of Course, you'd type his details again for every course he teaches. That's the repetition problem you already saw in 1.2.



Common Confusion

People try to put a relationship *inside* an entity.

Say you add a `course_taken` attribute to Student. Now what happens when Aisha takes six courses? You either need six columns, or you stuff six course codes into one cell. Both are a mess.

Here's the simple rule:

- A fact about **one** thing → attribute (ellipse hanging off a box)
- A fact that involves **two** things → relationship (diamond between two boxes)

One more small thing. The box labelled Student means the whole group of students — the category. Aisha is one student inside it. Books call the group an *entity set* and Aisha an *entity instance*. Everyday speech mixes the two; some interviewers won't.

Interview Angle

1. "What is an entity, and how do you decide if something should be one?" — It's a thing with its own identity and its own facts about it. Quick test: would it still be worth storing even if nothing else pointed to it?
2. "What's the difference between an attribute and a relationship?" — An attribute describes one thing; a relationship connects two. Give an example so it lands: name is an attribute, enrolls in is a relationship.
3. "Why draw an ER diagram instead of just writing the tables?" — Because mistakes are free to fix on paper and expensive to fix once data exists. Also, anyone can read a diagram — including the college staff who need to confirm you understood them.

Recap: An ER model draws the real world as entities (boxes), their attributes (ellipses, key underlined), and relationships between them (diamonds) — all before any table exists.

2.2 Cardinality & ER Notation — Core • 14 min

Hook

Two diagrams sit in front of you. Both say *Student* — *enrolls in* — *Course*. They look identical.

One of them needs **three** tables to build. The other needs **two**.

Nothing in either picture tells you which is which. Something is missing.

Why + What

What's missing is **how many**.

A diamond only says "these two are connected." It doesn't say whether one student connects to one course or to fifty. And that single fact decides how many tables you end up building. So a relationship without it isn't finished — it's half a drawing.

Cardinality means: for one instance of this entity, how many instances of the other entity can it connect to? There are three cases — one-to-one (1:1), one-to-many (1:N), and many-to-many (M:N).

How It Works (with the easiest example)

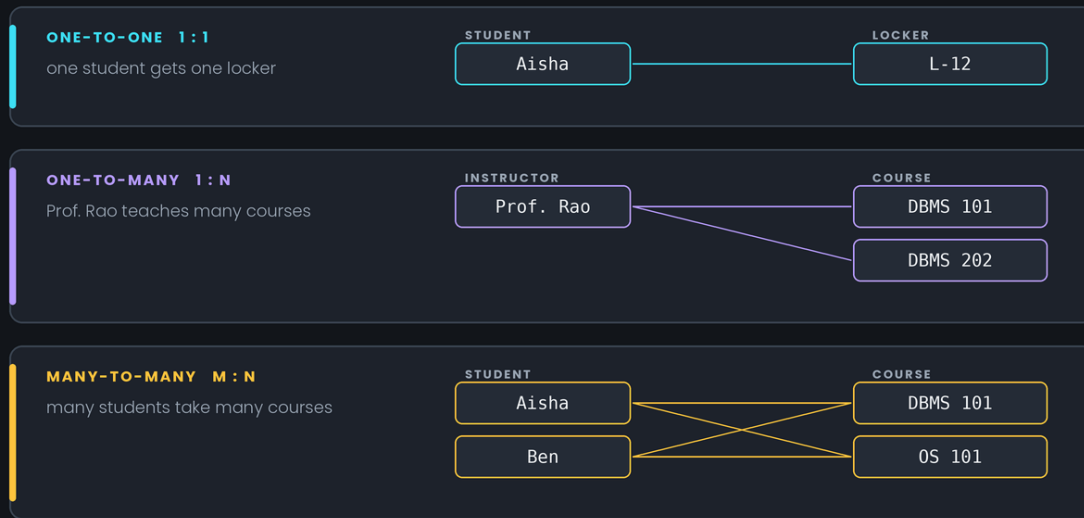
You already know how to draw boxes and diamonds. Now we add numbers to the line between them.

For each case, ask the same question in **both** directions.

1. **1:1 — Student and Locker.** One student gets one locker. One locker holds one student. Both answers are "one." So: Aisha ↔ locker L-12.
2. **1:N — Instructor and Course.** Prof. Rao teaches DBMS 101 *and* DBMS 202 — that's many. But DBMS 101 has exactly one instructor — that's one. One direction says many, the other says one. **This is the most common case in real systems.**
3. **M:N — Student and Course.** Aisha takes DBMS 101 and OS 101. And DBMS 101 has both Aisha and Ben in it. Many in both directions.
4. **Mark it on the line.** ER diagrams write 1 and N on the connecting line. Some styles draw a split "crow's foot" on the many side instead. Same meaning.
5. **See what it costs you.** 1:1 and 1:N fit in two tables — the "many" side just stores a pointer back to the "one" side. **M:N does not fit in two tables.** You need a third table in the middle. That's exactly why your Enrollment table exists: it's the bridge for a many-to-many.

Visualize

2.2 CARDINALITY



Notice: count the lines leaving one box — that is the cardinality.

Common Confusion

Direction confuses almost everyone.

"One-to-many" isn't a label for the whole relationship. It depends on which side you read from. Instructor → Course is 1:N. The exact same relationship read as Course → Instructor is N:1. Nothing changed except your starting point.

So don't argue about which one is right. Just say the direction out loud — "one instructor teaches many courses" — and the confusion disappears.

Second thing people mix up: **how many** vs **is it required**.

- **Cardinality** = how many? (one, or many)
- **Participation** = is it compulsory? (must have one, or can have none)

A student might have **zero** lockers. Still 1:1 for cardinality — but optional for participation. Interviewers like this one, because most candidates treat them as the same thing.

Interview Angle

1. "How would you implement a many-to-many relationship?" — With a third table in the middle (called a junction or bridge table), holding a pointer to each side: Enrollment(roll_no, course_code). Say *why* too: neither original table can hold a list of unknown length.
2. "Give a real 1:1 example — and why would you even split it into two tables?" — Student and locker. Or a user and their login password. Reasons to split: keep sensitive data separate, or keep rarely-used bulky columns out of the main table.

3. "Difference between cardinality and participation?" — Cardinality is how many, participation is whether it's required. One example each beats any definition.

Quiz

Q1. One author writes many books, and each book has exactly one author. This is: (a) 1:1 (b) 1:N (c) M:N (d) N:N **Answer: (b)** — many on the book side, one on the author side.

Q2. Which case *forces* you to create an extra table? (a) 1:1 (b) 1:N (c) M:N (d) none of them
Answer: (c) — M:N needs a junction table, because neither side can store a list of unknown length.

Q3 (scenario). A hospital tells you: "a patient stays in one room at a time, but a room hosts many patients over the year." What's the cardinality, and which table stores the pointer?
Answer: 1:N from Room to Patient — one room, many patients. The pointer (room_id) goes in the **Patient** table, which is the "many" side. Handy rule: **the many side always holds the pointer.**

Recap: Cardinality (1:1, 1:N, M:N) says how many instances connect, and it decides your table count — only M:N forces a third, junction table.

2.3 Weak Entities, Generalization & Specialization — Core • 14 min

Hook

Aisha's grade is **A**.

Now delete her enrollment in DBMS 101, but keep the grade.

What is that A now? A grade in what? Given to whom?

Why + What

It's nothing. Just a floating letter.

Some things in your model simply can't stand alone. If you pretend they can, you end up with leftover rows that point to nothing.

There's an opposite problem too. Look at Student and Instructor. Both store a name. Both store an email. You've now written the same two facts twice, in two places.

Two problems, two fixes:

A weak entity can't be identified by its own facts alone. It needs a parent entity to give it identity.

Generalization is the opposite move: take facts that two entities share, and lift them up into one shared parent — so they're written once.

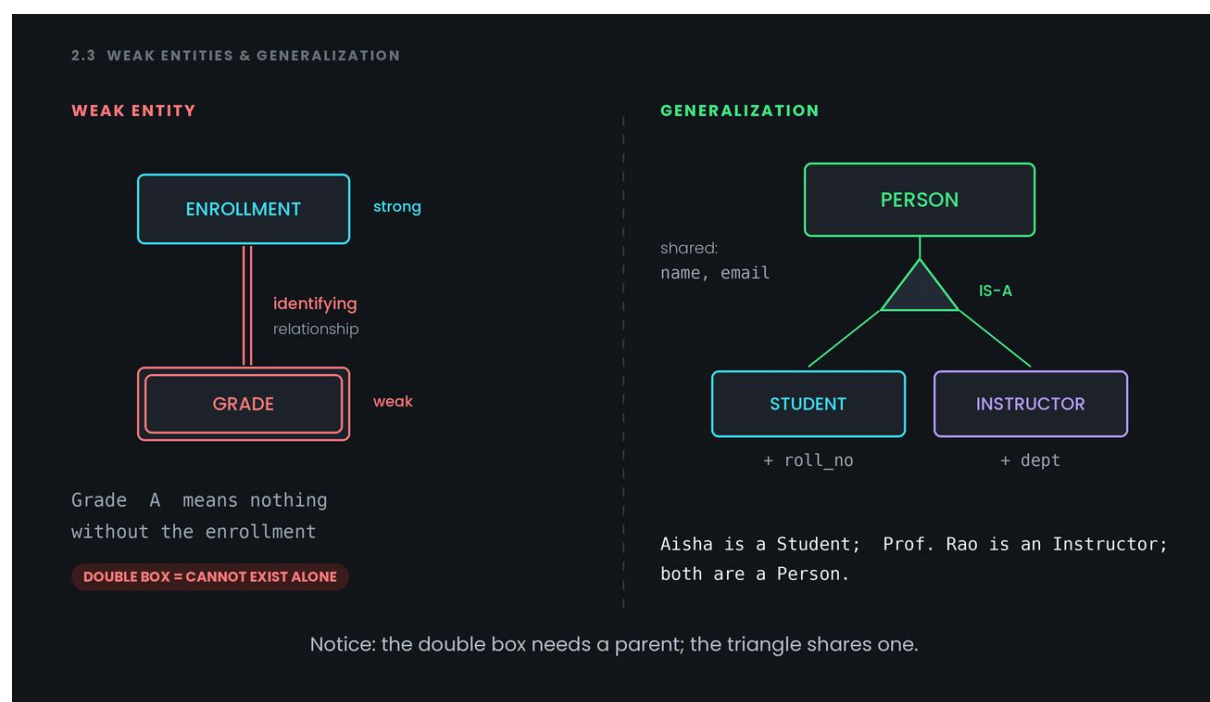
One is about **depending** on something. One is about **sharing** something. Both are about not repeating yourself.

How It Works (with the easiest example)

You've just learned cardinality. Now let's use these two ideas on our own data.

1. **Spot the one that can't stand alone.** Grade has one fact: the letter. "A" doesn't identify anything — hundreds of grades are "A". It has no key of its own. So Grade is a **weak entity**.
2. **Give it a parent.** Grade hangs off Enrollment. That link is called an **identifying relationship**. In the drawing, the weak entity gets a **double box**, and the link gets a **double line**. Delete the enrollment, and the grade goes with it.
3. **Build its identity.** Grade's full identity = the parent's key + a small piece of its own (called a **partial key**, shown with a dashed underline). So a grade is identified as *(roll 101, DBMS 101) + which grade slot it is*. Never as just "A".
4. **Now the opposite problem.** Student and Instructor both have a name and an email. Lift those two facts into a new parent called Person. Leave behind only what's different: roll_no stays in Student, dept stays in Instructor. That move is **generalization**.
5. **Read the link.** The parent connects to its children with an **IS-A** link, drawn as a **triangle**. Aisha IS-A Person. Prof. Rao IS-A Person. And if you'd started with Person and split it downward instead, that's called **specialization** — same drawing, opposite thinking.

Visualize



Notice: the double box needs a parent; the triangle shares one.

Common Confusion

People think "weak entity" means "an entity with a pointer to another table." It doesn't. Almost every entity has a pointer somewhere — that alone doesn't make it weak.

The real test is about **identity**, not pointers. Ask: *can I identify this thing without its parent's key?*

- Enrollment points to Student, but if it has its own enrollment ID, it can identify itself. **Not weak.**

- Grade cannot identify itself at all. **Weak.**

Generalization vs specialization is the other muddle, and here's the honest answer: **they produce the exact same drawing.** The only difference is which direction you were thinking.

- You had Student and Instructor, noticed shared facts, made Person → **generalization** (bottom-up)
- You had Person, split it into Student and Instructor → **specialization** (top-down)

Don't spend more time on it than that. Just be able to say which direction you took.

Interview Angle

1. *"What is a weak entity? Give an example."* — It has no key of its own, so it's identified using its parent's key plus a partial key. Example: Grade depending on Enrollment. Or a line item depending on an order.
2. *"Generalization vs specialization?"* — Same structure, opposite direction. Bottom-up: merge shared facts into a parent. Top-down: split a parent into subtypes. Mention that the diagram is identical — that shows you actually understand it instead of memorizing it.
3. *"If generalization means an extra join to get a full record, why do it?"* — Give the honest trade-off: it stops you defining the same facts twice, and keeps shared rules in one place. The cost is a join. Naming the cost is what makes this answer good.

Recap: A weak entity borrows its identity from a parent (double box + partial key), while generalization lifts shared facts into a common parent linked by IS-A — depending versus sharing.

Module 3: Relational Model

Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**.

3.1 Relations, Tuples & Schema — Core • 12 min

Hook

You've drawn boxes, ellipses and diamonds on paper. Nice.

Now the database asks a rude question: *"That's a picture. Where's my table?"*

Why + What

A drawing can't store anything. It has to turn into something real — and in a relational database, "real" means **one thing only: a table**.

That's the surprising part. Everything you drew in Module 2 — students, courses, the enrollment link between them — all of it becomes tables. There is no second shape. No special box for relationships. Just tables.

A relation is a table. Each row in it is a tuple. Each column is an attribute. The table's design — its name and column list — is the schema. The rows sitting in it right now are the instance.

Five words for things you already recognise. The only reason to learn the formal words is that interviewers use them.

How It Works (with the easiest example)

Let's turn the Student box from Module 2 into a real table.

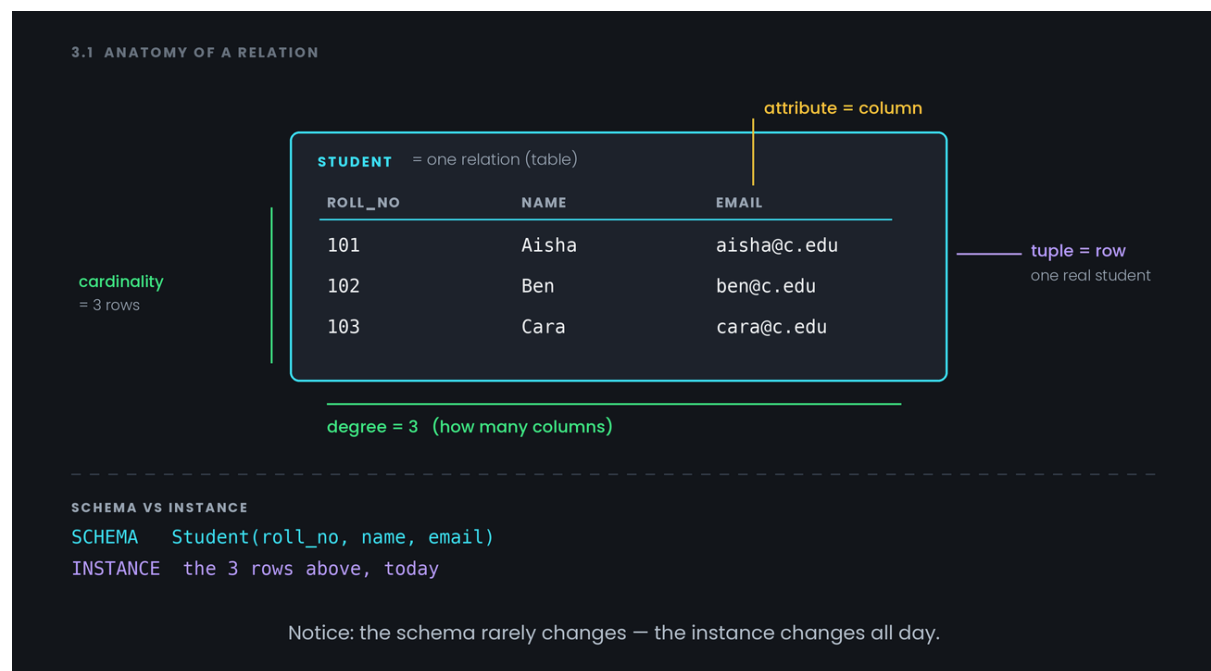
1. **The box becomes a relation.** Student is now a table. In formal language, a relation. Same thing — use whichever word the room uses.
2. **Each attribute becomes a column.** roll_no, name, email. Three columns.
3. **Each real student becomes a tuple.** Aisha's row: 101, Aisha, aisha@c.edu. One tuple = one row = one real thing.
4. **Count the columns → degree.** Three columns, so the degree is 3. This changes only if you redesign the table.
5. **Count the rows → cardinality.** Three students today, so cardinality is 3. Add one student and it's 4.

Careful with that last word. **Cardinality here means "number of rows"** — not the 1:N kind from Module 2.2. Same word, two jobs. Which one is meant is always clear from context: rows, or relationships.

Pause here. Your table has 5 columns and 200 rows. What's the degree, and what's the cardinality?

Degree 5, cardinality 200. Columns first, rows second.

Visualize



Notice: the schema rarely changes — the instance changes all day.

Common Confusion

Schema vs instance is the one that costs marks.

- **Schema** = the design. `Student(roll_no, name, email)`. Written once, changes rarely.
- **Instance** = the actual rows in the table right now. Changes every time anyone inserts, updates or deletes.

Simple way to hold it: the schema is the empty form, the instance is the filled-in copies. And you already met this idea in 1.3 — the schema lives at the conceptual level, and it's what data independence protects.

One more small thing. In theory, a relation is a **set** of tuples, so no two rows should be identical and rows have no fixed order. Real SQL is looser — it will happily let you store duplicate rows if you don't stop it. If an interviewer asks "can a relation have duplicate rows," the theory answer is no, and the practical answer is "SQL allows it unless a key prevents it." Saying both scores better than saying either.

Interview Angle

1. *"What's the difference between a schema and an instance?"* — Schema is the structure, instance is the current data. Add that the schema is stable and the instance changes constantly.
2. *"Define degree and cardinality of a relation."* — Degree = number of columns. Cardinality = number of rows. Worth flagging that "cardinality" also means something different for relationships, so you know both uses.

3. *"Can a relation contain duplicate tuples?"* — In relational theory, no, because a relation is a set. In real SQL, yes, unless a primary key or unique constraint blocks it.

Quiz: skipped — reinforced above.

Recap: A relation is a table of tuples (rows) and attributes (columns) — its design is the schema, its current rows are the instance, degree counts columns and cardinality counts rows.

3.2 Keys — Core • 16 min

Hook

Two students walk into the office. Both are called Aisha. Both are in second year. Both study DBMS 101.

You need to give one of them a grade. Which one?

Why + What

You can't tell them apart, so you can't do anything safely. Not update, not delete, not grade. Every row in a table needs some way to be picked out with zero doubt.

Names don't do it. Nor does a combination of ordinary facts, since two people can share all of them. You need something guaranteed unique.

A key is a column, or a set of columns, whose values are always unique — so any one row can be picked out exactly.

There are a few kinds. They sound like five separate ideas but they're really one idea with filters applied.

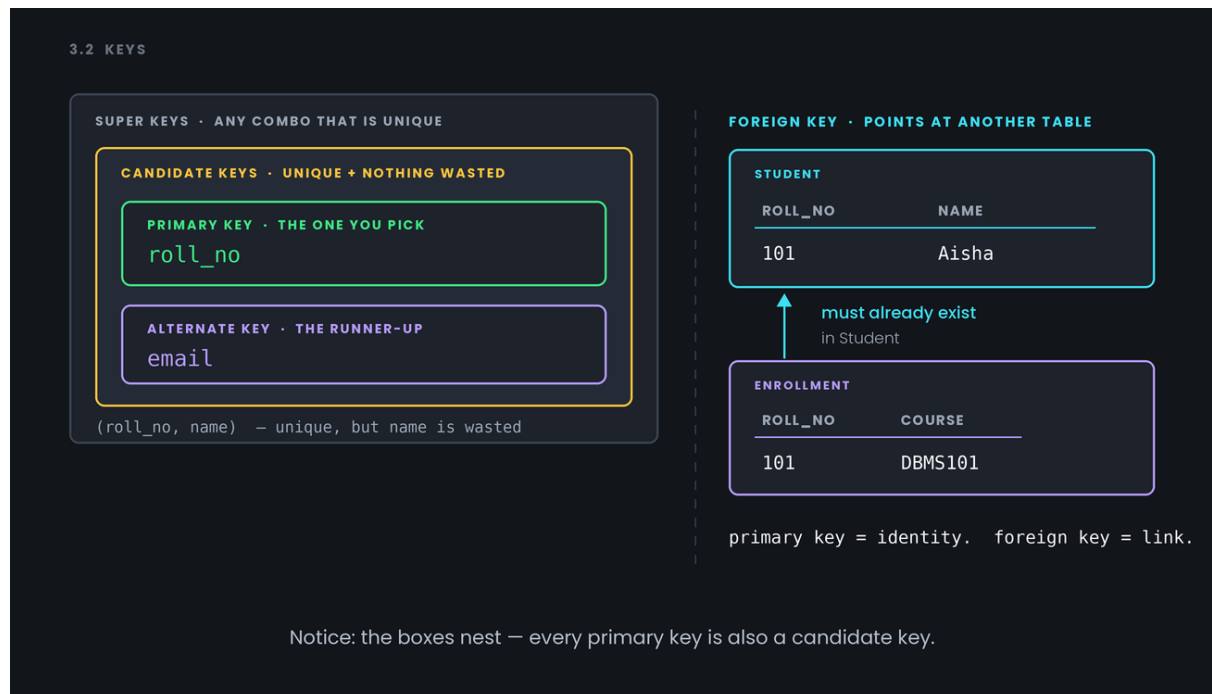
How It Works (with the easiest example)

You already know what a table and a tuple are. Now let's find the keys in Student(roll_no, name, email).

1. **Start wide — super key.** Any set of columns that happens to be unique is a **super key**. roll_no is one. So is (roll_no, name). So is (roll_no, name, email). All unique — but the last two are carrying dead weight, because roll_no alone was already enough.
2. **Trim the dead weight — candidate key.** A **candidate key** is a super key with nothing spare in it. Remove any column and it stops being unique. Here we have two: roll_no, and email. Both identify a student on their own.
3. **Pick one — primary key.** You choose one candidate to be the official identifier. That's the **primary key**. We pick roll_no. It can never be empty and never repeat.
4. **The leftovers — alternate key.** email was a perfectly good candidate but didn't get the job. That makes it an **alternate key**. It's still unique — just not the official one.
5. **Point at another table — foreign key.** In Enrollment, the column roll_no isn't identifying the enrollment. It's pointing at a row in Student. That's a **foreign key**. You met this in 1.2: it's what makes the database reject roll 999.

One line to hold all five: **super key** = unique. **Candidate key** = unique and minimal. **Primary key** = the candidate you chose. **Alternate key** = the ones you didn't. **Foreign key** = a pointer to someone else's primary key.

Visualize



Notice: the boxes nest — every primary key is also a candidate key.

Common Confusion

The big one: **super key vs candidate key**.

Both are unique. The difference is whether anything in there is useless.

- (roll_no, name) — unique, so it's a super key. But name is doing no work. **Not** a candidate key.
- roll_no — unique, and you can't remove anything. **Candidate key**.

So every candidate key is a super key, but not the other way around. That's exactly why the diagram nests the boxes.

Second confusion: people think a **composite key** is its own separate category. It isn't. "Composite" just means the key is made of more than one column — like Enrollment(roll_no, course_code), where you need both together to identify one enrollment. A composite key can be a candidate key or a primary key. It's a description of shape, not a new type.

Third: **can a foreign key be NULL?** Yes, usually — a NULL foreign key just means "this row doesn't point at anything yet." A student with no locker assigned. A **primary key can never be NULL**. People mix these up constantly, so the contrast is worth memorising: primary key never empty, foreign key may be empty.

Interview Angle

1. *"Difference between a super key and a candidate key?"* — Both unique; a candidate key has nothing extra in it. Give the (roll_no, name) vs roll_no example — the example is what proves you get it.
2. *"Can a table have more than one candidate key? More than one primary key?"* — Many candidate keys, yes (roll_no and email). Only ever **one** primary key, though it may be built from several columns.
3. *"Can a foreign key be NULL? Can a primary key?"* — Foreign key yes (nothing linked yet). Primary key never — a row with no identity can't be found or updated.
4. *"How would you pick between two candidate keys?"* — Prefer the one that's short, never changes, and is never unknown at insert time. Roll number beats email, because people change their email.

Quiz

Q1. (roll_no, name) is unique in the Student table. It is: (a) a candidate key (b) a super key only (c) a foreign key (d) not a key at all **Answer: (b)** — it's unique, so it's a super key, but name is unnecessary, so it fails the minimal test for a candidate key.

Q2. How many primary keys can one table have? (a) exactly one (b) one per column (c) as many as it has candidate keys (d) zero or more **Answer: (a)** — exactly one, though that one key may be made of multiple columns.

Q3 (scenario). In Enrollment(roll_no, course_code, grade), neither roll_no nor course_code is unique on its own — Aisha appears twice, DBMS 101 appears many times. What's the primary key, and what are those two columns *also* doing? **Answer:** The primary key is the **composite** (roll_no, course_code) — together they're unique, since Aisha enrolls in DBMS 101 only once. And each column is separately a **foreign key**, pointing at Student and Course. So the same columns are doing two jobs at once, which is completely normal in a junction table.

Recap: A key identifies rows uniquely — super key (unique), candidate key (unique and minimal), primary key (the chosen candidate, never NULL), alternate key (the unchosen ones), foreign key (a pointer to another table's primary key).

3.3 Integrity Constraints — Core • 14 min

Hook

Someone types a grade of **"banana"**.

Someone else adds a student with **no roll number**.

A third person records a grade for **roll 999**, who doesn't exist.

Nobody is being malicious. It's a Tuesday. Who stops them?

Why + What

Back in 1.1 you learned that a DBMS is storage **plus enforced rules**. This topic is those rules, by name.

They're worth taking seriously because bad data doesn't announce itself. It sits quietly until a report is wrong, and then nobody can tell which rows to trust.

An integrity constraint is a rule the database enforces on every insert, update and delete. If a change would break the rule, the change is refused.

Three constraints matter for interviews.

How It Works (with the easiest example)

You know what primary and foreign keys are now. These rules are built on them.

1. **Domain integrity — is this value even allowed?** Every column has a type and a set of permitted values. grade accepts A, B, C. So "banana" is refused. So is a birth date of 32-13-2026.
2. **Entity integrity — does this row have an identity?** The primary key can't be NULL and can't repeat. A student row with no roll_no is refused, because you'd never be able to find or update it again. Insert roll 101 twice and the second one is refused too.
3. **Referential integrity — does the thing I'm pointing at exist?** A foreign key must match a real row in the other table, or be NULL. Enrollment with roll_no = 999 is refused, because no such student exists.
4. **Then someone deletes Aisha.** Her enrollments still point at roll 101. What now? You choose the behaviour up front:
 - **CASCADE** — delete her enrollments too
 - **RESTRICT** — block the delete while enrollments exist
 - **SET NULL** — keep the enrollments but blank out the link
5. **Notice what all three have in common.** None of them fix the data or guess your intent. They refuse the change and tell you. That's the whole design: better a rejected insert than a quietly wrong database.

Visualize

3.3 INTEGRITY CONSTRAINTS

DOMAIN INTEGRITY

each value must fit its column's type / allowed set

✗ "banana" is not a grade
the DBMS rejects the row

ROLL_NO	NAME	GRADE
101	Aisha	A
104	Dev	banana

ENTITY INTEGRITY

the primary key can never be empty or repeated

✗ no roll_no = no identity
the DBMS rejects the row

ROLL_NO	NAME	GRADE
101	Aisha	A
NULL	Eve	B

REFERENTIAL INTEGRITY

a foreign key must point at a row that exists

✗ roll 999 is not a student
the DBMS rejects the row

ROLL_NO	COURSE	GRADE
101	DBMS101	A
999	DBMS101	A

Notice: all three do the same job — refuse the row, keep the data true.

Notice: all three do the same job — refuse the row, keep the data true.

Common Confusion

People mix up **entity** and **referential** integrity, because both involve keys. Split them by which table the problem is in:

- **Entity integrity** — about **this** row's own primary key. Is it present and unique?
- **Referential integrity** — about a row in **another** table. Does the thing my foreign key points to exist?

Inside vs outside. That's the whole difference.

Second one: NULL is not zero, and not an empty string. **NULL means "no value here."** A grade of 0 means the student scored zero. A grade of NULL means nobody has entered a grade yet. Two very different facts, and treating them as the same is how averages come out wrong.

Interview Angle

1. "What are the main integrity constraints?" — Domain (valid values), entity (primary key not NULL and unique), referential (foreign key must point at something real). One quick example each is better than three definitions.
2. "What happens if you delete a row another table points at?" — It depends on the rule you set: CASCADE deletes the children, RESTRICT blocks the delete, SET NULL clears the link. Say that the choice is made when you design the table, not when the delete happens.
3. "Difference between entity and referential integrity?" — Entity is about this row's own primary key; referential is about whether the row you point at exists. Inside versus outside.

4. *"Why does the database refuse bad data instead of just correcting it?"* — Because correcting means guessing your intent, and a wrong guess spreads silently. A rejected insert is visible; a quietly wrong row isn't.

Quiz

Q1. Inserting a student with roll_no = NULL breaks: (a) domain integrity (b) entity integrity (c) referential integrity (d) nothing **Answer: (b)** — a primary key can never be NULL, so the row would have no identity.

Q2. You delete a course that enrollments still point at, and the enrollment rows vanish automatically. The rule in force is: (a) RESTRICT (b) SET NULL (c) CASCADE (d) domain integrity **Answer: (c)** — CASCADE passes the delete down to the child rows.

Recap: Integrity constraints are rules enforced on every change — domain (valid values), entity (primary key present and unique), referential (foreign key points at a real row) — and a change that breaks one is refused, not fixed.